

FACULDADE COTEMIG
CIÊNCIA DA COMPUTAÇÃO

ANA CAROLINA RODRIGUES DE ALMEIDA CÉSAR ALVES

THIAGO FERRAZ WERNECK XAVIER

FUNDAMENTOS TEÓRICOS DA COMPUTAÇÃO
TRABALHO FINAL - IMPLEMENTAÇÃO DE MÁQUINAS ABSTRATAS:
AFD, AUTÔMATO DE PILHA E MÁQUINA DE TURING

Belo Horizonte - MG

2026

1. INTRODUÇÃO

A Teoria da Computação é uma área da Ciência da Computação que estuda os limites do que pode ser computado e os modelos formais capazes de representar processos computacionais. Entre estes modelos, destacam-se os Autômatos Finitos, Autômatos de Pilha e as Máquinas de Turing.

Este trabalho tem como objetivo implementar e analisar três modelos da Teoria da Computação: o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) e a Máquina de Turing (MT). Além da implementação, busca compreender suas diferenças estruturais e capacidades de reconhecimento de linguagens formais.

Para colocar esses conceitos em prática, desenvolvemos simuladores genéricos utilizando a linguagem de programação C#. A ideia principal foi traduzir os formalismos matemáticos (como as tuplas, estados e funções de transição) diretamente para o código estruturado. Com isso, o relatório busca apresentar o comportamento de cada uma dessas máquinas ao processarem os arquivos de testes obrigatórios, analisando como o acréscimo de memória (como a pilha no AP e a fita na MT) altera completamente o poder de processamento de cada modelo computacional.

2. FUNDAMENTAÇÃO TEÓRICA

2.1. AUTÔMATOS FINITOS DETERMINÍSTICOS

Um Autômato Finito Determinístico (AFD) é usado em processamento em processamento de texto, compiladores e projeto de hardware (SIPSER, 2007), eles são modelos matemáticos para reconhecer linguagens regulares. A principal característica de um AFD é o seu determinismo, para cada estado atual e para cada símbolo lido do alfabeto de entrada, existe exatamente uma única transição possível para um próximo estado, não havendo margem para ambiguidades ou escolhas simultâneas.

Formalmente, um AFD é definido como uma quintupla $M = (Q, \Sigma, \delta, q_0, F)$, onde Q é um conjunto de estados, Σ é o alfabeto de entrada, δ é a função de transição, o q_0 é o estado inicial e o F é o estado final (NASCIMENTO, 2024).

2.1.1. Exemplo de Cálculo Manual (AFD)

Cadeia "bab":

- $\delta(q_0, 'b') = q_0$
- $\delta(q_0, 'a') = q_1$
- $\delta(q_1, 'b') = q_2$

Resultado: A leitura terminou no estado $q_2 \in F$. Cadeia ACEITA.

Cadeia "ba":

- $\delta(q_0, 'b') = q_0$
- $\delta(q_0, 'a') = q_1$

Resultado: A leitura terminou no estado $q_1 \notin F$. Cadeia REJEITA.

2.2. AUTÔMATOS DE PILHA

Os Autômatos de Pilha (AP) são modelos computacionais semelhantes aos autômatos finitos não-determinísticos, porém com uma estrutura adicional de memória chamada pilha (HOPCROFT, 2002). Essa pilha funciona como uma memória extra, já que o controle do autômato sozinho é finito. Com isso, conseguem reconhecer linguagens mais complexas do que as regulares.

Além disso, os autômatos de pilha possuem a mesma capacidade computacional das gramáticas livres de contexto.

Essa relação é útil porque permite duas abordagens para demonstrar que uma linguagem pertence a essa classe: pode-se construir uma gramática livre de contexto que gere ou um autômato com pilha que a reconheça. Dependendo da linguagem, uma dessas representações pode ser mais simples ou mais natural do que a outra (SIPSER, 2007).

2.2.1. Exemplo de Cálculo Manual (AP)

Cadeia "aabb":

- $(q_0, 'aabb', [Z_0])$ = Configuração inicial com base da pilha.
- $(q_0, 'abb', [A, Z_0])$ = Lê 'a', empilha 'A'.
- $(q_0, 'bb', [A, A, Z_0])$ = Lê 'a', empilha 'A'.
- $(q_1, 'b', [A, Z_0])$ = Transiciona para q_1 ao ler 'b', desempilha 'A'.
- $(q_1, '', [Z_0])$ = Lê 'b', desempilha 'A'.
- $(q_2, '', [])$ = Por meio de uma ϵ -transição, consome o símbolo Z_0 .

Resultado: A fita acabou e a pilha está vazia. Cadeia **ACEITA**.

Cadeia "aab":

- $(q_0, 'aab', [Z_0])$
- $(q_0, 'ab', [A, Z_0])$
- $(q_0, 'b', [A, A, Z_0])$
- $(q_1, '', [A, Z_0]) = \text{Lê 'b', desempilha o primeiro 'A'}$.

Resultado: Como a cadeia terminou, mas os elementos $[A, Z_0]$ permanecem retidos na pilha, o critério de esvaziamento falha. Cadeia **REJEITA**.

2.3. Máquina de Turing

Segundo Sipser (2007), a máquina de Turing, proposta por Alan Turing em 1936, funciona como um modelo teórico preciso de um computador de propósito geral. Diferente dos autômatos finitos, ela possui uma memória ilimitada e irrestrita baseada em uma fita infinita, na qual uma cabeça de leitura e escrita manipula símbolos e se movimenta livremente. Embora esse modelo seja capaz de realizar qualquer operação que um computador real execute, ele também estabelece os limites teóricos da computação, demonstrando a existência de problemas que não podem ser resolvidos.

Formalmente, o autor define a máquina de Turing como uma 7-upla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, composta pelos seguintes elementos:

- Q é o conjunto finito de estados;
- Σ é o alfabeto de entrada ($\sqcup \in \Sigma$);
- $\Gamma \supseteq \Sigma$ é o alfabeto da fita ($\sqcup \in \Gamma$);
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ é a função de transição;
- q_0 é o estado inicial;
- q_{accept} é o estado de aceitação;
- q_{reject} é o estado de rejeição ($q_{\text{reject}} \neq q_{\text{accept}}$).

2.3.1. Exemplo de Cálculo Manual (MT)

Cadeia "abc":

- $q_0abc_ = \text{Estado inicial apontando para o primeiro caractere}$.

- $Xq_1bc_$ = Substitui 'a' por 'X', move para a direita, muda para q_1 .
- $XYq_2c_$ = Encontra 'b', substitui por 'Y', move para a direita e muda para q_2 .
- $XYZq_3_$ = Encontra 'c', substitui por 'Z', move para a direita e muda para q_3 .
- $XYq_4Z_$ = Encontra o espaço vazio '_', move para a esquerda para verificar se restam caracteres desemparelhados.

Resultado: Após retornar e validar que todos os grupos foram devidamente marcados e limpos, o fluxo atinge o estado final q_{accept} . Cadeia **ACEITA**.

Cadeia "aab":

- $q_0ab_$
- $Xq_1b_$ = Encontra e marca 'a'.
- XYq_2 = Encontra e marca 'b'. Avança procurando o elemento 'c'.

Resultado: No estado q_2 , a máquina lê o caractere em branco '_' sem ter encontrado o correspondente 'c'. A transição leva a fita diretamente para o estado de erro q_{reject} . Cadeia **REJEITA**.

3. DESCRIÇÃO DA IMPLEMENTAÇÃO

As principais decisões do projeto envolveram a definição da arquitetura do sistema, a organização dos seus componentes e a escolha das estruturas de dados mais adequadas para a implementação. O objetivo principal foi traduzir os modelos matemáticos abstratos em programas funcionais e bem estruturados. Para isso, utilizou-se a linguagem C# em conjunto com a plataforma .NET 6 (ou superior), desenvolvendo toda a lógica independente, sem o uso de bibliotecas externas de automação.

O ambiente de desenvolvimento foi organizado seguindo estritamente a estrutura de diretórios exigida na especificação do trabalho. Isso garantiu que as responsabilidades de cada máquina abstrata ficassem completamente isoladas:

```
ftc-20261-final/  
├── Parte1/  
│   ├── config/  
│   ├── data/  
│   ├── models/  
│   ├── services/  
│   ├── Parte1.csproj  
│   └── Program.cs  
├── Parte2/  
│   ├── data/  
│   ├── models/  
│   ├── services/  
│   ├── Parte2.csproj  
│   └── Program.cs  
├── Parte3/  
│   ├── config/  
│   ├── data/  
│   ├── models/  
│   ├── services/  
│   ├── Parte3.csproj  
│   └── Program.cs  
├── .gitattributes  
├── .gitignore  
└── README.md
```

O papel de cada componente na arquitetura global do sistema é definido da seguinte forma:

3.1. PARTE 1 (AUTÔMATO FINITO DETERMINÍSTICO)

- **config/afd.json**: Arquivo de configuração responsável por armazenar a definição formal do AFD (estados, alfabeto, estado inicial, estados de aceitação e transições) para permitir a carga dinâmica.

- **data/entradas.txt**: Arquivo de texto contendo as cadeias de caracteres que devem ser testadas pelo simulador, organizadas uma por linha.
- **models/**: Reúne as classes de dados. **Estado.cs** e **Transicao.cs** definem os elementos básicos. **AFD.cs** consolida a representação interna da 5-tupla matemática. **AfdJsonModel.cs** e **TransicaoJsonModel.cs** servem como estruturas auxiliares para mapear os dados do arquivo JSON.
- **services/**: **FileReaderService.cs** gerencia a leitura do arquivo de texto. **AfdConfigLoaderService.cs** faz o *parsing* do JSON e **AfdSimulatorService.cs** executa a simulação passo a passo, gerando o rastro de estados visitados.
- **Program.cs**: Ponto de entrada do console da Parte 1, responsável por inicializar os serviços e rodar os testes.

3.2. PARTE 2 (AUTÔMATO DE PILHA)

- **data/entradas_ap.txt**: Arquivo de texto contendo as cadeias de teste para as linguagens L_2 e L_3 .
- **models/**: Contém a classe **AutomatoPilha.cs**, que modela a 7-tupla formal e gerencia a memória do tipo LIFO. A classe **TransicaoAP.cs** trata as regras de movimentação, incluindo os ϵ -movimentos (transições vazias).
- **services/**: **ApSimulatorService.cs** é o motor que executa as transições e imprime em tempo real a configuração instantânea exigida (estado, cadeia restante e conteúdo da pilha). **FileReaderAPService.cs** realiza a leitura das entradas.
- **Program.cs**: Arquivo principal que dispara os testes do Autômato de Pilha por critério exclusivo de pilha vazia.

3.3. PARTE 3 (MÁQUINA DE TURING)

- **config/mt.json**: Guarda as definições de estados e a tabela de transições determinísticas para as duas variações da máquina (reconhecadora e computadora).
- **data/entradas_mt.txt**: Contém as palavras de teste para o reconhecimento de L_4 e os valores em formato unário para a função computada.

- **models/**: A classe **MaquinaTuring.cs** modela os estados e os componentes formais da 7-tupla. **TransicaoMT.cs** dita a regra de transição baseada em leitura, escrita e direção do cabeçote (L, R).
- **services/**: **MtSimulatorService.cs** implementa a fita dinâmica, controla a posição do cabeçote e atualiza a exibição passo a passo na tela. Também controla o contador de passos para evitar loops infinitos. **MtConfigLoaderService.cs** faz a leitura das configurações.
- **Program.cs**: Executa as simulações e exibe as mensagens de aceitação, rejeição ou o resultado final gravado na fita.

3.4. ARQUIVOS DA RAIZ DO REPOSITÓRIO

- **.gitignore** e **.gitattributes**: Arquivos de configuração que evitam o envio de pastas temporárias de compilação para o GitHub e garantem a correta padronização do código.
- **README.md**: Documento de texto explicativo que serve como página inicial do repositório, contendo a identificação do grupo, instruções de execução e o link do vídeo.

O projeto foi feito utilizando Programação Orientada a Objetos (POO). A padronização dos nomes das subpastas (**data**, **models**, **services**) entre as três partes foi adotada com o intuito de garantir a organização arquitetural e a manutenibilidade do projeto de acordo com as regras de avaliação.

4. RESULTADOS E TESTES

Para validar o funcionamento dos simuladores desenvolvidos, foram realizados testes práticos utilizando os arquivos de entrada com as cadeias sugeridas pela especificação. Os resultados obtidos no console foram listados e organizados nas tabelas abaixo, divididas por cada parte do projeto.

Máquina	Cadeia de Entrada	Rastro / Conteúdo Final da Pilha	Resultado
AFD (L_1)	ab	$q_0 - a \rightarrow q_1 - b \rightarrow q_2$	ACEITA

AFD (L_1)	aab	$q_0 -a \rightarrow q_1 -a \rightarrow q_1 -b \rightarrow q_2$	REJEITA
AFD (L_1)	ba	$q_0 -b \rightarrow q_0 -a \rightarrow q_1$	REJEITA
AFD (L_1)	ϵ	q_0 (sem movimentos)	REJEITA
AP (L_2)	ab	ϵ	ACEITA
AP (L_2)	aabb	ϵ	ACEITA
AP (L_2)	ba	ZA	REJEITA
AP (L_2)	ϵ	Z	REJEITA
AP (L_3)	a	ϵ	ACEITA
AP (L_3)	aba	ϵ	ACEITA
AP (L_3)	ab	Zab	REJEITA
AP (L_3)	aab	ϵ (Cadeia sobrou / Erro)	REJEITA
MT (L_4)	abc	_XYZ_	ACEITA
MT (L_4)	aabbcc	_XXYYZZ_	ACEITA
MT (L_4)	aabbc	_XXYYZ	REJEITA
MT (L_4)	abc abc	_XYZ[]abc	REJEITA
MT (unário)	1	11	
MT (unário)	111	1111	
MT (unário)	11111	111111	

5. ANÁLISE COMPARATIVA

A classificação das capacidades computacionais dos simuladores desenvolvidos fundamenta-se na Hierarquia de Chomsky. De acordo com Hopcroft, Motwani e Ullman (2002), o poder de reconhecimento de uma

máquina abstrata é determinado diretamente pela sua estrutura de transições e pelo modelo de memória auxiliar disponível.

5.1. AUTÔMATO FINITO DETERMINÍSTICO (AFD)

O AFD representa o modelo de menor poder computacional devido a ausência de uma memória secundária. O armazenamento de informações é estritamente limitado ao seu conjunto finito de estados internos Q .

5.2. AUTÔMATO DE PILHA (AP)

O Autômato de Pilha estende as capacidades do AFD ao introduzir uma memória auxiliar de tamanho ilimitado estruturada como uma pilha. Contudo, o acesso a essa estrutura é restrito ao topo (LIFO).

5.3. MÁQUINA DE TURING (MT)

A Máquina de Turing constitui o modelo definitivo de computação clássica, sendo considerada a melhor formalização da noção de algoritmo na história da matemática. Sua fita dinâmica ilimitada funciona como uma memória de acesso aleatório bidirecional, permitindo que o cabeçote leia, escreva e navegue livremente para a esquerda e direita.

5.4. COMPARATIVA DE PODER COMPUTACIONAL

Propriedade / Critério	Autômato Finito (AFD)	Autômato de Pilha (AP)	Máquina de Turing (MT)
Memória Auxiliar	Nenhuma (Apenas estados)	Pilha infinita (Acesso LIFO)	Fita ilimitada (Acesso aleatório)
Movimentação do Cabeçote	Unidirecional (Apenas avança)	Unidirecional (Consome entrada)	Bidirecional (Esquerda e Direita)

Classe de Linguagem	Linguagens Regulares	Livres de Contexto	Sensíveis ao Contexto / Enumeráveis
O que pode reconhecer	Padrões fixos sem contagem. Ex: Cadeias terminadas em ab (L_1)	Contagens simples ou espelhadas. Ex: $a^n b^n$ (L_2) e Palíndromos (L_3)	Múltiplas contagens e funções. Ex: $a^n b^n c^n$ (L_4) e $f(n) = n + 1$
O que NÃO pode reconhecer	Linguagens com contagem ou memórias dinâmicas (Ex: L_2 , L_3 , L_4)	Contagens múltiplas/independentes ou restrições cruzadas (Ex: L_4)	Problemas indecidíveis ou sem algoritmo lógico (Problema da Parada)
Modo de Operação	Reconhecedor	Reconhecedor	Reconhecedor e Transdutor

6. QUESTÕES REFLEXIVAS

6.1. PARTE 1

6.1.1. POR QUE A LINGUAGEM L_1 É REGULAR? APRESENTE A EXPRESSÃO REGULAR EQUIVALENTE.

A linguagem L_1 é considerada regular porque ela pode ser descrita por uma expressão regular equivalente e reconhecida por um Autômato Finito Determinístico (AFD), sem precisar de

nenhuma memória extra para contar letras. O critério dela pede apenas para identificar um padrão fixo no final da palavra, o que o nosso código faz mudando entre estados simples toda vez que lê um caractere do alfabeto $\{a, b\}$. Como a regra aceita qualquer combinação de letras no começo, contanto que os dois últimos caracteres formem a sequência exata exigida, a expressão regular que representa essa linguagem foi definida no projeto da seguinte forma:

$$(a \mid b)^* ab$$

6.1.2. DESCREVA FORMALMENTE A 5-TUPLA DO AFD CONSTRUÍDO, JUSTIFICANDO A ESCOLHA DE CADA ESTADO.

- **$Q = \{q_0, q_1, q_2\}$** : É o conjunto finito de estados necessários para rastrear o sufixo da cadeia. O estado q_0 representa a situação inicial e funciona como a base quando nenhuma parte do padrão final foi identificada (ou após ler um 'b' fora de contexto). O estado q_1 foi escolhido para sinalizar que a máquina acabou de ler a letra 'a', indicando que o padrão começou a ser formado. O estado q_2 foi escolhido para representar o sucesso da leitura da sequência exata 'ab'.
- **$\Sigma = \{a, b\}$** : É o alfabeto oficial da linguagem, contendo os únicos símbolos válidos para o processamento.
- **q_0** : É o estado inicial por onde o simulador começa a leitura de qualquer cadeia de entrada.
- **$F = \{q_2\}$** : É o conjunto de estados de aceitação. Apenas o estado q_2 é final, garantindo que a palavra só seja aceita se a execução terminar exatamente nele.
- **δ** : É a função de transição que dita as mudanças de estado.

6.1.3. O QUE ACONTECERIA SE A FUNÇÃO δ NÃO FOSSE TOTAL? COMO VOCÊ TRATOU ENTRADAS INVÁLIDAS (SÍMBOLOS FORA DO ALFABETO)?

Se a função de transição δ não fosse total, significaria que haveria estados sem nenhuma transição mapeada para certas letras do alfabeto. Na teoria, se o autômato estivesse em um estado e lesse uma letra sem transição definida, ele travaria ou "morreria", rejeitando a cadeia imediatamente por não conseguir terminar a leitura. No código do simulador, para evitar travamentos ou erros de execução, esse problema foi tratado de forma preventiva. Antes de disparar qualquer mudança de estado, o programa faz uma checagem em um laço `foreach` ou bloco `if` para validar se o caractere faz parte do alfabeto $\Sigma = \{a, b\}$. Se o usuário digitar uma entrada inválida (como a letra 'c' testada nos resultados), o simulador aborta a execução na hora, exibe uma mensagem de erro no console e define o resultado como rejeitado.

6.2. PARTE 2

6.2.1. POR QUE L_2 NÃO PODE SER RECONHECIDA POR UM AFD? DEMONSTRE USANDO O LEMA DO BOMBEAMENTO PARA LINGUAGENS REGULARES.

A linguagem $L_2 = \{ a^n b^n \mid n \geq 1 \}$ não pode ser reconhecida por um AFD porque exige que a máquina conte a quantidade de letras a para depois garantir que leu exatamente a mesma quantidade de letras b. Como o AFD possui apenas um conjunto fixo e finito de estados, ele não tem uma memória auxiliar (como uma pilha) para armazenar essa contagem quando o valor de n cresce ao infinito.

Para demonstrar pelo Lema do Bombeamento, supõe-se que L_2 seja regular e que exista uma constante de bombeamento p. Escolhe-se então a palavra $w = a^p b^p$, que pertence à linguagem pois seu tamanho é maior ou igual a p. O lema garante que essa palavra pode ser dividida em três partes, $w = xyz$, respeitando

as regras $|xy| \leq p$ e $|y| > 0$. Como a parte xy está obrigatoriamente dentro do primeiro bloco de tamanho p , a string y contém apenas letras 'a' (ou seja, $y = a^k$ com $k \geq 1$). Ao bombear a palavra para $i = 2$, gerando xy^2z , o novo formato da string passa a ser $a^{p+k}b^p$. Como $p+k$ é diferente de p , a nova palavra modificada não pertence a L_2 , gerando uma contradição e provando que a linguagem não é regular.

6.2.2. A ACEITAÇÃO POR PILHA VAZIA E POR ESTADO FINAL SÃO EQUIVALENTES EM PODER DE RECONHECIMENTO? DEMONSTRE CONCEITUALMENTE.

Os dois critérios são totalmente equivalentes em poder de reconhecimento. Isso significa que qualquer linguagem que um Autômato de Pilha (AP) aceita por estado final, outro AP também consegue aceitar por pilha vazia, e vice-versa. Mudar o critério não deixa a máquina mais forte ou mais fraca.

- Do Estado Final para Pilha Vazia: Se o autômato original aceita chegando a um estado final, nós podemos criar um estado novo focado só em "limpar" a máquina. Sempre que o programa atingir um estado de aceitação, nós puxamos uma transição vazia para esse novo estado, que vai desempenhar tudo o que sobrou até a pilha ficar totalmente vazia.
- Do Pilha Vazia para Estado Final: e a máquina original valida a cadeia quando a pilha esvazia, nós adicionamos um marcador especial bem no fundo da pilha (um Z_0 extra) antes de começar a leitura. Se o programa rodar a palavra inteira e tentar desempilhar esse marcador, o código entende que a pilha original teria chegado ao fim. Nesse momento, o fluxo é jogado direto para um novo estado que definimos como o estado final de aceitação.

6.2.3. EXPLIQUE O PAPEL DO SÍMBOLO Z_0 NA SUA IMPLEMENTAÇÃO.

O símbolo Z_0 atua como o marcador de fundo de pilha no simulador. Como a pilha inicia vazia, a primeira ação do programa antes de ler a cadeia de entrada é empilhar esse caractere. Ele serve basicamente como um ponto de referência para a máquina identificar o momento exato em que a pilha foi totalmente esvaziada.

Na linguagem $L_2 = \{ a^n b^n \}$, o algoritmo insere letras a sobre esse Z_0 . Quando o fluxo muda para as letras b, o código começa a desempilhar os a's correspondentes. Se a palavra estiver no padrão correto, após processar o último b, o topo da pilha exibirá novamente o Z_0 . O simulador então realiza uma transição final para remover esse marcador. Como o critério de aceitação é por pilha vazia, ver a pilha completamente limpa após tirar o Z_0 é o sinal de que o programa precisa para validar a cadeia.

6.3. PARTE 3

6.3.1. POR QUE L_4 NÃO PODE SER RECONHECIDA POR UM AUTÔMATO DE PILHA? QUAL PROPRIEDADE DA MT POSSIBILITA O SEU RECONHECIMENTO?

A linguagem L_4 não pode ser reconhecida por um Autômato de Pilha porque exige a checagem de mais de uma contagem ao mesmo tempo ($a^n b^n c^n$). Como a pilha funciona na lógica LIFO (o último que entra é o primeiro que sai), quando a gente desempilha os dados para conferir as letras b, a informação se perde e não sobra histórico para validar as letras c.

A propriedade que permite à Máquina de Turing (MT) resolver isso é o acesso bidirecional de leitura e escrita na fita. Diferente da pilha, a MT consegue mover seu cabeçote para a esquerda e para a direita livremente. Isso deixa o algoritmo marcar uma letra no começo, andar até o final para conferir a outra e voltar para o início sem apagar ou destruir os dados do caminho.

6.3.2. QUANTOS ESTADOS FORAM NECESSÁRIOS PARA A MT DE L_4 ? DESCREVA A ESTRATÉGIA DE “MARCAÇÃO” ADOTADA.

Para a Máquina de Turing de L_4 ($a^n b^n c^n$), foram necessários 6 estados no total. A estratégia de marcação funcionou por meio de um movimento de ida e volta controlado por símbolos substitutos. O algoritmo lê o primeiro a disponível, troca ele por um X e muda de estado para buscar o b correspondente. Ele ignora os outros a 's no caminho, acha o primeiro b , troca por um Y e passa a buscar o c . Ao encontrar o primeiro c , troca por um Z . Depois disso, a máquina inicia o retorno, movendo o cabeçote para a esquerda até achar o último X marcado. O ciclo recomeça a partir dali para o próximo grupo de letras, garantindo que a quantidade de todas seja idêntica até que não reste nenhuma letra original na fita.

6.3.3. UM COMPUTADOR MODERNO É MAIS PODEROSO QUE UMA MÁQUINA DE TURING? JUSTIFIQUE À LUZ DA TESE DE CHURCH-TURING.

Um computador moderno não é mais poderoso que uma Máquina de Turing (MT) em termos de capacidade de computação. À luz da Tese de Church-Turing, qualquer algoritmo ou problema que pode ser resolvido por um computador de última geração também pode ser resolvido por uma MT padrão, pois ambas as estruturas definem o que é "computável".

A grande diferença prática está apenas na eficiência. Um computador atual tem processadores extremamente velozes, barramentos paralelos e memória de acesso aleatório (RAM), o que o torna infinitamente mais rápido. A MT realiza as mesmas tarefas, mas de forma mecânica e lenta, movendo uma fita de memória infinita passo a passo. Portanto, em termos de poder de resolução, elas são equivalentes.

7. CONCLUSÃO

O desenvolvimento dos simuladores em C# foi importante para entender na prática os fundamentos da teoria de autômatos e linguagens formais. Programar o comportamento do AFD, do AP e da Máquina de Turing ajudou a compreender visualmente a Hierarquia de Chomsky, deixando claro como a presença e o tipo de memória mudam a capacidade de processamento e o poder de reconhecimento de cada máquina estudada.

A implementação prática dos simuladores permitiu acompanhar o passo a passo das transições de estados, os empilhamentos e desempilhamentos no autômato de pilha, e a movimentação bidirecional com gravação na fita da Máquina de Turing por meio dos logs do console. A execução de todos os casos de testes propostos serviu para observar o comportamento prático das máquinas em cada cenário, ajudando no desenvolvimento do raciocínio lógico e algorítmico exigido na disciplina.

8. REFERÊNCIAS

HOPCROFT, J.; MOTWANI, R.; ULLMAN, J. **Introdução à Teoria de Autômatos, Linguagens e Computação**. Boston: Elsevier, 2002. Disponível em:

<https://www.livros1.com.br/pdf-view/introdu%C3%A7%C3%A3o-%C3%A0-teoria-dos-aut%C3%B4matos--linguagens-e-c%C3%A1lculo.pdf>. Acesso em: 08 jun. 2026.

NASCIMENTO, Guilherme Augusto Anício Drummond do. **Elaboração de algoritmos e criação de um simulador gráfico para o ensino de autômatos de pilha**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Instituto de Ciências Exatas e Biológicas, Universidade Federal de Ouro Preto, Ouro Preto, 2023. Disponível em: https://monografias.ufop.br/bitstream/35400000/6479/3/MONOGRAFIA_Elabora%C3%A7%C3%A3oAlgoritmosCria%C3%A7%C3%A3o.pdf. Acesso em: 06 jun. 2026.

SIPSER, Michael. **Introdução à Teoria da Computação**. Tradução da 2. ed. norte-americana. São Paulo: Cengage Learning, 2007. <<https://www.kufunda.net/publicdocs/Introdu%C3%A7%C3%A3o%20%C3%A>

[0%20Teoria%20da%20Computa%C3%A7%C3%A3o%20%28Michael%20Sipser%29.pdf](#)>. Acesso em: 06 jun. 2026.